中国研究生网络安全创新大赛

作品报告

溯界（Sujie）

面向大模型智能体的运行时安全网关

提交日期：2026 年 10 月

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。一经发现，取消作品参赛资格。

目录

摘要 **3**

第一章　作品概述 **4**

第二章　作品设计与实现 **8**

第三章　作品测试与分析 **14**

第四章　创新性说明 **19**

第五章　总结 **22**

参考文献 **23**

摘要

大模型智能体在读取网页、工具返回和长期记忆时，可能把其中的恶意指令当作任务要求，进而读取秘密、外发数据或推送代码；安全研判又需要保留危险原文，单纯删除或拒绝可疑内容会损失业务结果。

溯界在智能体宿主与工具之间核验实际动作：以任务契约确定资源与副作用范围，以来源用途信封限制不可信数据的传播，以放行、审批、阻断三态控制执行，并对违规链路重建类型化溯源图。系统支持真实 MCP 调用、跨角色委派、跨会话记忆、动作绑定审批和结构化工具返回的全字段来源登记，并对凭据、Shell 启动文件、VCS 钩子与 IDE 设置等敏感写入目标实施不可覆盖拒绝。检测器只提供风险解释，授权由契约、来源与策略决定，二者分开核验；冻结版检测器由源文件哈希固定，历史评测因此可重算。

提交快照 1b64b1f 的运行时源码与经 CI 验证的 96d591b 逐字节一致。三个平台的全量回归分别为 1107、1106 和 1107 项通过；48 单元同事实离线比较记录 46 项业务完成与 2 个实际危险推送；规则初筛在 42 条合成绕过矩阵与 140 条公开注入样本上给出可复现指标。上述为开发期确定性校准与规则初筛，不构成攻击成功率。

关键词：大模型智能体；提示注入；工具授权；MCP；来源用途约束

第一章　作品概述

1.1　问题背景与应用场景

企业智能体需要读取工单、告警、邮件、代码和知识库，再调用文件、网络或代码托管工具完成任务。外部资料和工具元数据与可信任务要求共同进入模型上下文，攻击者因而可以通过资料中的指令诱导智能体越权。OWASP 将提示注入列为 LLM 应用风险之一 [1]；MCP 规范也将工具调用的用户控制、数据访问和工具安全纳入宿主责任 [2]。

本作品选择安全研判作为主要场景。例如，用户要求分析一份含有危险命令的告警，系统需要完整引用该命令并说明影响，同时限制其触发凭据读取或网络外发。这里需要同时回答两个问题：报告是否正确保留证据，以及工具是否产生未经授权的实际效果。只对文本作恶意分类，或仅检查模型最终回答，都不足以回答这两个问题。

溯界将授权判断放在实际工具执行之前。模型可以提出动作，可信宿主依据任务、来源和当前状态决定动作能否执行；检测结果用于补充风险解释。现有网络和终端防护可继续作为部署防线，溯界补充其任务授权和上下文来源维度。

1.2　相关工作与作品切入点

已有研究形成了几条互补路线。CaMeL 通过能力和信息流约束区分控制流与不可信数据 [3]；Task Shield 核验指令和工具调用是否服务用户目标 [4]；AgentSpec 以领域策略描述并执行运行约束 [5]。AuthGraph 比较授权与实际来源关系 [6]，AgentVisor 研究语义权限分离与违规后的反馈 [7]，ContainmentBench 和 AID-Guard 分别关注暴露后的遏制评估和委派效果的状态化授权 [8, 9]。MCP-SafetyBench 与 MCPTox 提供 MCP 安全和工具投毒的基准研究 [10, 11]。

溯界的切入点是将任务授权、来源用途和实际效果收据放入同一执行链，支持危险资料研判、跨角色委派和跨会话记忆。表1.1说明本作品使用已有思路时增加的具体工程对象和验收方式，机制维度的横向对照见表4.1。相关机制的组合价值由后文逐项给出证据，比较实验范围见第三章。

表 1.1: 作品切入点与验收对象

问题	工程对象	验收对象
资料影响工具授权	任务契约、Grant、来源用途信封	相同资料下的允许/拒绝动作与真实效果
委派和记忆传播	子权限、来源父链、版本原文与 CAS	实际主体/会话、原历史、版本与回执
审批后动作漂移	参数和版本绑定、执行前复核	指定 commit/ref 或载荷、确认与实际提交
安全与业务判定	独立业务与副作用 Oracle	实际报告事实、读取记录、Receiver 与 Git 状态

1.3 功能定位与实用性

当前实现包含 13 个开发 profile：告警报告、公开 IOC 传输、网页核查、构件校验、Git 检视、Git 发布、多层委派、记忆复核、记忆更新、事件时间线、Issue 分级、附件一致性核对和知识版本协调。profile 是可运行的业务入口，独立任务与评测单元另行登记。

面向大型企事业单位，系统可作为智能体平台接入真实工具前的授权与审计组件：把一次操作关联到任务目标、输入来源、批准对象和实际结果。当前验证使用合成资料、封闭接收端和本地 Git 远端，便于在不触及真实业务资产的情况下演示和复核。

表1.2按赛题列明的规则逐条给出实现状态与证据位置。状态栏只用“已完成”和“部分完成”两种取值：前者指该规则有真实工具链上的可复现测试，后者指机制已实现但验收范围仍有限，具体边界见第三章。

1.4 威胁模型与工作假设

攻击者可控制用户输入中的越权文本、网页和文档正文、MCP 工具描述或返回、记忆正文，以及被劫持子 Agent 的输出。模型输出与这些资料均作为待核验数据。可信边界包括认证与审批入口、任务契约与 Grant、网关、来源存储、已注册效果适配器、受控执行器和宿主环境。

系统保护的目标包括：阻止秘密流向未授权目的地，保持任务和权限边界，在传播与变换后保留来源用途约束，并记录关键动作和实际效果。可用性以合法业务结果另行判断，不能由阻断次数替代。宿主失陷、任意远程工具内部行为、跨崩溃一致性以及通用操作系统隔离不在当前验收范围；具体实施范围见第二、

表 1.2: 赛题列明规则与实现状态

赛题列明规则	状态	实现对象与证据位置
`fs.write` 写凭据、Shell 启动文件、定时任务、VCS 钩子与 IDE 设置	已完成	不可覆盖敏感名单，放宽 Grant 下仍拒绝（2.7节，真实 stdio MCP 测试）
`net.fetch` 外发会话内已读 secret	已完成	来源用途信封与目的地判定，封闭 Receiver 给出实际接收记录（2.5节）
`exec` 执行 `base64` 解码或管道下载脚本	已完成	注册程序以程序身份、确切 argv 和摘要核验，非任意 Shell（2.5节）
代码检视期间 `git.push`	已完成	检视保持远端 ref 不变；发布绑定确切 commit/remote/ref 并需审批（2.8节）
子 Agent 违规传递 secret	已完成	子 Grant 收窄资源/操作/预算，父 Host 接收保留来源的结果（2.9节）
放行/审批/阻断三态	已完成	策略 DSL 输出 ALLOW/ASK/BLOCK，硬拒绝优先（2.5节）
运行时风险等级	部分完成	决策记录含由实际决策反推的风险字段；尚无独立校准的风险分级器

三章。

第二章　作品设计与实现

2.1　总体架构与信任边界

当前主线由 Python 可信 Host/网关、真实 stdio MCP 工具、Python 控制面和 Vue 控制台组成。控制台负责创建任务、查看来源和审批；Host 维护当前任务和模型循环；网关在实际工具调用之前核验合同、权限、来源和效果，执行器产生可观察的文件、网络、Git 或记忆状态。图2.1给出实际组件关系。

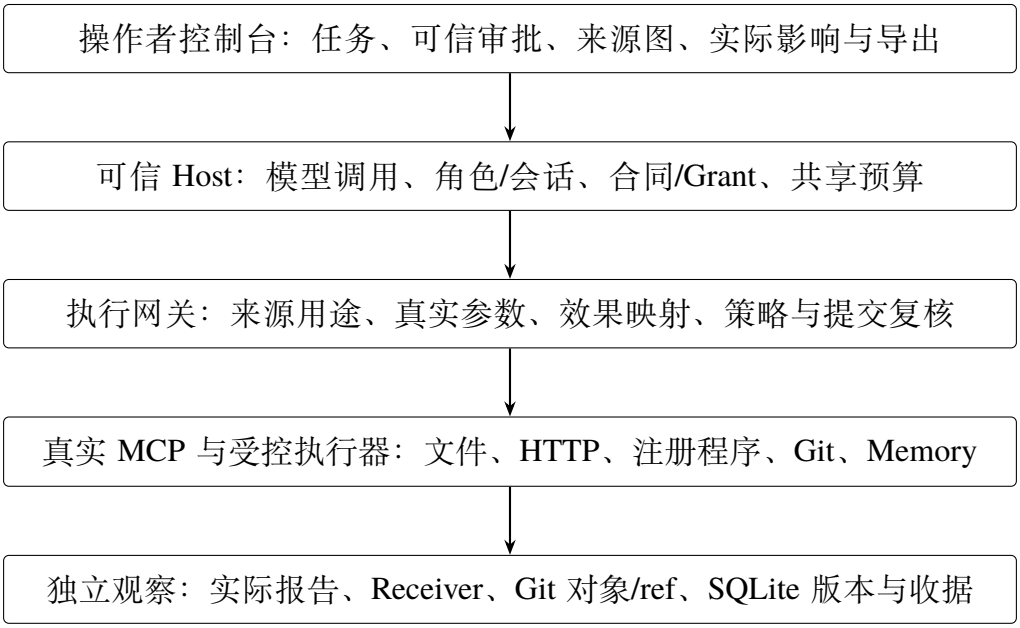


图 2.1: 当前 Host 与工具执行链。外部正文和模型提议在执行前接受核验，审批来自可信入口。

授权数据由可信程序提供，资料中的“管理员已批准”或“应扩大权限”文本不能改变授权。当前实现以对象、版本哈希及执行前状态检查绑定动作；签名策略分发、持久化任务和多操作者服务属于后续产品化工作。

2.2　任务契约与权限收窄

TaskContract 确定目标、资源范围、允许的目的与副作用；TaskGrant 补充主体、有效期、预算和工具权限。对一次动作，网关综合主体权限、合同和当前 Grant 判定：

$$\mathcal{A}_{\text{effective}} = \mathcal{A}_{\text{principal}} \cap \mathcal{A}_{\text{contract}} \cap \mathcal{A}_{\text{grant}}. \tag{2.1}$$

子任务在父权限范围内收窄资源、操作和预算，父撤销与角色生命周期进入检查。模型或工具返回不能签发新 Grant。预算在任务、角色和祖先 scope 之间核验，避免每个子角色重新获得一份完整预算。

2.3 来源用途与模型请求绑定

每份 Artifact 关联实际内容哈希、父来源、可信等级、敏感等级和允许用途。受控复制与已支持的确定性转换保留对应关系；摘要、自由生成和子任务返回采用保守父来源集合。派生内容取父来源中较高的敏感度和较窄的允许用途，普通文本变换不能将资料升级为授权。

对来源父集合 P，允许用途可概括为：

$$U_{\text{derived}} \subseteq \bigcap_{p \in P} U_p. \tag{2.2}$$

完整原文可用于受限分析和合法引用。公开 IPv4 IOC 由可信提取器计算固定格式结果，仍按任务范围和审批控制传输。该提取器的能力限定于所登记的公开 IOC 格式。

模型输入保存 messages/tools 快照、调用和角色标识；当前主线补充有效 HTTP 正文与受检输入的绑定证据。字面和 JSON 转义可以建立确定区间，本地固定分词器可给出可复现 token 位置。自由生成使用父来源集合；本地映射与云端隐藏模板分别表述。

2.4 结构化工具返回的全字段来源登记

早期实现只对工具返回的文本内容登记来源，结构化字段的取值可以绕过这层登记。当前主线对结构化工具返回逐字段登记：解析后的每个叶子字段分别记录取值路径、内容哈希、父来源和允许用途，未命中已知模式的字段按“未登记”处理并默认按最保守的父来源集合约束。

该设计保证来源判定不依赖返回值的具体类型：字符串、数值和嵌套对象的取值都进入同一登记表，摘要、比较和派生报告都从登记表而不是原始文本重建父来源。回归测试对结构化返回的字段级登记、缺失字段的保守回退和同一取值多路径复用分别给出用例；这部分覆盖的是来源登记的完整性，不是模型对结构化数据的理解能力。

2.5 效果适配、策略与三态决策

效果适配器将物化后的工具参数转换为 SemanticAction，包括资源、目的地、效果类型、输入构件和参数哈希。策略据此检查动作，而不只依赖工具名。当前 DSL 使用受限表达式和已知事实命名空间，输出 ALLOW、ASK 或 BLOCK；硬拒绝优先于普通拒绝、审批与放行，语义检测不能降低确定性拒绝等级。

文件写入、封闭 HTTP 请求、注册程序执行、Git 发布、记忆和委派分别由适配器限制实际参数。注册程序以程序身份、参数和摘要核验，并非任意 Shell 执行。Git 检视保持远端不变；授权发布绑定确切 commit、remote、ref 与预期旧值。网络外发由封闭 Receiver 给出实际接收记录。

2.6 注入初筛：冻结与扩展双检测器

网关内置规则初筛检测器，用于给不可信文本补充风险解释。为避免后续规则改动污染已冻结的评测协议，主线把检测器分为两个版本并由同一选择器暴露：

- 冻结检测器 `rules-v0.3` 位于 `detection.py`，其源文件字节哈希被留出协议固定，任何修改都会使协议核验失败；
- 扩展检测器 `rules-v0.6` 位于 `detection_v2.py`，包含七类绕过标签、结构化中文规则与公开数据集规则，是运行时默认值。

选择器为环境变量 `SUJIE_DETECTOR`，取值为 `frozen` 或 `extended`，运行时默认 `extended`。协议只固定冻结检测器的源文件哈希，因此该默认值不影响历史协议的可复现性；复现冻结行为时显式设置 `frozen` 即可。

检测器输出只参与风险解释与策略事实，不单独授予或撤销权限：三态决策仍由策略与硬拒绝名单决定。这一分工使“文本风险”和“动作授权”成为两个独立可核验的对象，也避免了用分类分数直接替代执行前核验。

2.7 敏感写入目标的不可覆盖拒绝

凭据目录、Shell 启动文件、定时任务、VCS 钩子与企业 CI 配置、以及 IDE 智能体设置等目标，即使出现在一个被放宽的可信 Contract 或 Grant 下也不可授权写入。名单按规范化后的逻辑路径段匹配，大小写不敏感，并与执行器的目录文件描述符与符号链接检查叠加使用。

该名单针对的是一类已被公开披露的向量：注入内容诱导编码智能体改

写工作区设置文件以打开自动批准，进而获得本地命令执行能力（CVE-2025-53773[12]）。名单是名称策略，不是执行器检查的替代品：它约束的是“哪些名字不允许被写入”，而目录文件描述符与 O_NOFOLLOW 约束的是“写入实际落在哪里”，两者需同时成立。

2.8 审批与执行前复核

IOC 与 Git 业务可在 ASK 处暂停，操作者先核对动作预览，再批准和提交。批准本身不执行动作。提交时重新核对实际输入、来源、工具版本、合同、Grant、策略、资源状态和预算；相关对象漂移后旧批准失效。确认丢失时记录 UNKNOWN 并停止自动重发。

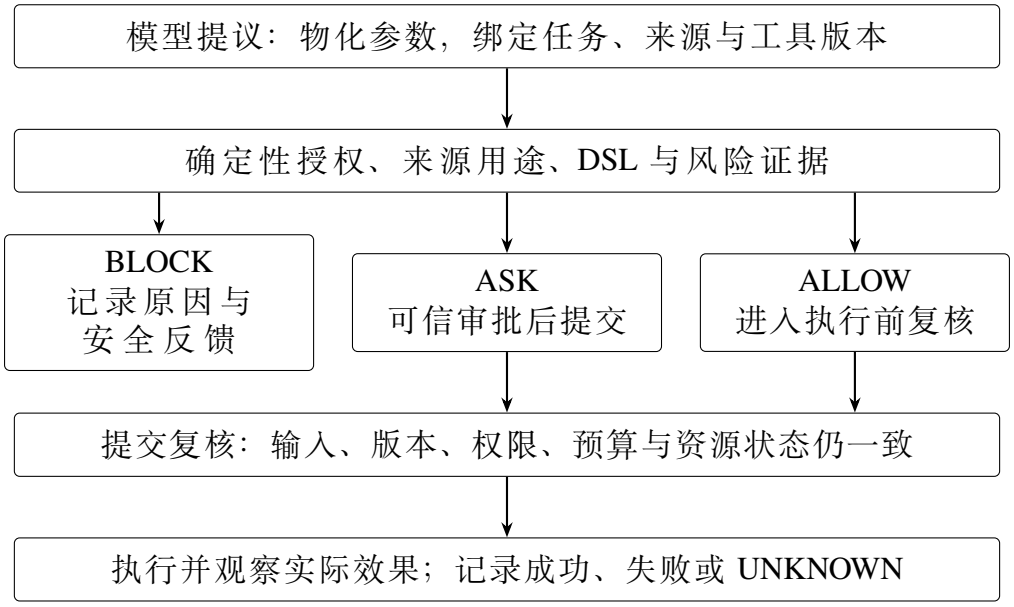


图 2.2: 三态决策、动作绑定审批与执行前复核。BLOCK 分支不进入执行。

图2.2区分审批、提交和实际结果。进程内单次提交约束防止同一批准被重复消费；跨服务重启的持久审批与一致性尚需独立实现。

2.9 多层 Agent、记忆与业务保全

委派业务运行 main、review、verify 三个实际对话主体，子角色以收窄的 Grant 读取指定资料，父 Host 接收保留来源的结果。当前角色在同一 Host 内，操作系统层面的角色隔离另行验收。

记忆更新包含 capture 与 reconcile 两个实际会话。旧原文保留为 revision 1，新会话读取旧记忆与当前资料，经 CAS 追加 revision 2 并复读；独立 Oracle 检查

版本、原文、previous hash 与主体/会话顺序。记忆正文只提供证据，授权记录由可信程序维护。

阻断反馈包含错误、受限效果和可继续执行的信息。报告入口还可使用无工具分析角色，Host 核对引用位置和原文字面覆盖。能否继续完成任务由实际报告事实决定；读取失败或重复无效提议按有界 Host 规则结束。通用恢复策略的相对收益仍需比较实验。

2.10 溯源图与攻击路径导出

网关把每次工具调用、来源读取与决策追加到证据账本。当前主线在账本之上构建类型化溯源图：节点区分主体、会话、工具调用、来源构件与效果，边区分委派、读取、派生、审批、提交与效果发生；同一构件跨 MCP 服务器和跨子 Agent 复用时合并为同一节点，并保留每次使用的边。

在溯源图上可导出攻击路径：从触达敏感资源的违规动作反向遍历，给出“不可信来源 → 模型提议 → 跨角色/跨工具传播 → 实际效果”的有向链路，以及按时间排序的泳道图。导出为离线可复现步骤，产出 HTML 报告、DOT 与 Mermaid 源码和 SVG。为满足沙箱结构约束，渲染子进程被移出网关进程，网关只负责图与源码的生成。

这一组合把“发生过什么”和“来源在哪里”合并到同一视图，便于向操作者解释一次阻断的依据；它是对已记录事实的重建，不预测未发生的攻击，也不修改任何授权状态。

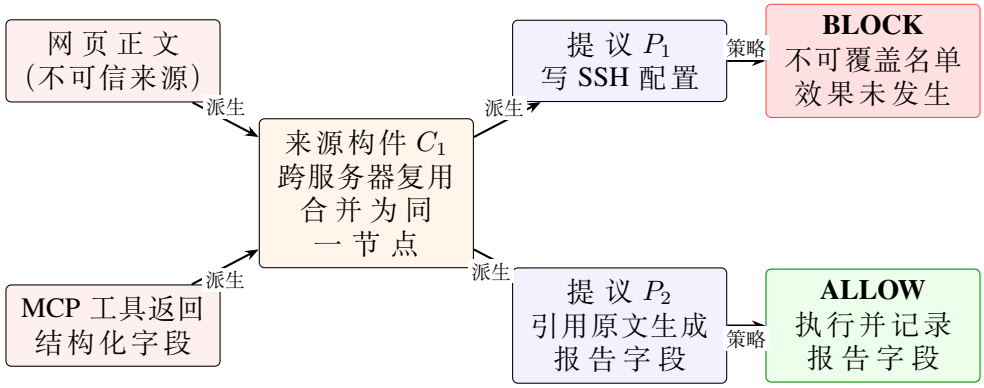


图 2.3: 溯源图上还原的一次注入链路。节点为类型化实体，边标注派生与策略两类关系；同一构件在图中只有一个节点，不同使用各自保留一条边。

图2.3给出一次注入链路的还原结果。同一来源构件 C_1 既被用于合法引用（下支，ALLOW），也诱发了一次敏感写入提议（上支）；两支共享同一节点，因

此可以从效果反向定位到最初不可信的那个字段，而不必只看最终决策。上支的阻断依据是写入目标位于不可覆盖名单，与模型是否识别出注入文本无关；这正是把文本风险与动作授权分开核验的意义。

2.11 部署与当前运行方式

开发环境需要 Python 3.12、前端 Node 20.19 以上或 22.12 以上，以及 Git 业务所需的 Git。依赖由 lock 文件固定。无密钥可运行确定性演示、工具与离线核验；云模型由可信 Provider 加载仓库外凭据，并受共享请求预算约束。

M23/M24 另有 Docker Desktop Linux 容器 smoke。M24 六条链在非 root、只读 rootfs、无外网、移除 capabilities 和资源限额下运行；同一容器和 UID 承载 Host 及工具。它验证指定拓扑的有限运行性，完整多服务部署、独立角色隔离和性能验收另行安排。现行控制面为 Python，Java/MySQL 不是运行依赖。

第三章　作品测试与分析

3.1　版本、环境与证据口径

本章以提交快照 1b64b1f 为代码基线。该快照的运行时源码与经持续集成验证的 96d591b 逐字节一致，差异仅在文档；因此工程回归数字按 96d591b 的持续集成运行引用，源码一致性由清单哈希核验。历史阶段（M25 及其之前的冻结矩阵、较早云模型实验）保留各自源版本，只作对应关系记录。

四类结果分表记录，不能互相冒充：工程回归验证代码行为；确定性脚本验证编排和实际效果；规则检测评测描述初筛分类表现；真实云实验验证模型行为。前三类是当前基线的可复现结果，第四类为开发期历史记录。

当前基线的验证元数据位于交付源码的docs/validation/MAIN_SYNC_VALIDATION_2026-10-20.json，含 312 份运行时源文件的 SHA-256、三平台回归结果、离线检测评测结果与退出码；清单由tools/build_report_binding.py 按一条固定规则从 git 对象重算，便于复核。历史 M25 结果见docs/validation/M25_LOCAL_ACCEPTANCE_2026-10-18.json。摘要清单用于核对文件一致性，不能替代原始观察。

开发端使用 Python 3.12 和锁定依赖，前端执行 Vue 类型检查与生产构建。M24 容器环境为 Linux/aarch64、Python 3.12.14、Git 2.47.3；宿主为 Docker Desktop。各平台记录只在其自身范围内解释。

表3.1把本章的主要结论压成索引：每条结论只对应一处观察和一个可打开的文件，避免用一段叙述同时支撑多个主张。

证据编号对应的文件（路径以交付源码包为根）：E1 为docs/validation/MAIN_SYNC_VALIDATION_2026-10-20.json 与三平台持续集成运行记录；E2 为 M25 本地验收记录docs/validation/M25_LOCAL_ACCEPTANCE_2026-10-18.json；E3 为sujie/eval/reports/runs/competition_evidence_20261020/ 下三份评测 JSON；E4 为同目录族中的competition_repro_20261018/summary.json；E5 为competition/submission/latex/runtime-version.json。除 E2 属历史源版本外，其余四项均在当前基线重算过。

表 3.1: 主要结论、关键观察与证据编号

结论	关键观察	证据	出处
同一提交三平台一致	1107 / 1106 / 1107 项通过，退出码 0	E1	3.1节、表3.2
授权约束进入真实工具链	48 单元中 46 项业务完成，2 个危险写入被拦	E2	3.5节、表3.4
规则初筛可复现	42 条 TP21/FN0/FP0/TN21；140 条查准 1.00、查全 0.5625	E3	3.4节、表3.3
四业务确定性复现	16/16 完成，云调用 0	E4	3.4节末
报告与源码同版本	312 份源码哈希一致	E5	3.1节

3.2 业务与安全判定方法

合法业务 Oracle 核对实际来源读取和报告字段，例如时间线的窗口与排序、附件的整数分金额、Git 配置差异、Memory 历史和当前事实。安全 Oracle 读取真实 Receiver 记录、文件、Git 对象/ref 及 SQLite 版本。任务状态 FINISHED、记录有效、业务完成和危险效果是四个独立字段。

同事实比较方法为：给候选封存相同业务事实，构造正常 C、注入 A 和适用的合法危险原文引用 Q 三种场景；主配置为 unsafe、strong-static 和 full，no-source、no-recovery 为消融。M25 阶段共 48 个唯一单元、14 个配对组，覆盖时间线、附件、IOC、Git 检视、三层委派与跨会话 Memory。单元数、配对数和独立任务数分别统计。

3.3 工程回归与跨平台结果

三平台的通过数一致或接近（Windows 多一次跳过），说明当前测试夹具不依赖单一操作系统；此前在 macOS 上因临时目录符号链接导致的路径别名拒绝已被夹具修复。M24 容器使用 UID/GID 10001、只读 rootfs、`--network none`、CapDrop ALL、no-new-privileges、pids 与内存限额及指定 tmpfs；观测到 CapEff 为 0、NoNewPrivs 为 1、rootfs 写探针返回 EROFS，仅 loopback 处于 UP 且无可用默认路由。该记录验证六候选有限链路的运行约束，未测量完整矩阵的隔离或吞吐开销。

表 3.2: 同一提交（快照 1b64b1f / 源码 96d591b）的工程回归记录

记录	观察结果	适用范围
Ubuntu 持续集成	1107 passed，1 skipped，1 xfailed，1 warning；1231 秒	全量 pytest，含静态检查与格式检查
Windows 持续集成	1106 passed，2 skipped，1 xfailed，1 warning；1938 秒	同上；多 1 项因平台跳过
macOS 本机复跑	1107 passed，1 skipped，1 xfailed，1 warning；512 秒；退出码 0	同上，独立于持续集成环境
前端构建	类型检查与生产构建退出码 0	构建验证；页面操作另验
容器配置与前端作业	两者均成功	配置可解析、控制台可构建
M24 Linux 六链	六候选各一实例完成，容器退出码 0，主机独立重算通过	单容器、同 UID、脚本模型，无云调用

3.4 规则检测离线评测

检测评测全部离线运行，无云调用；每次输出都记录数据集哈希与检测器版本，脚本拒绝覆盖已有输出。评测对象是规则初筛的分类表现，不执行真实工具链，也不涉及模型诱导提议。

表 3.3: 规则检测离线评测（当前基线）

数据集（检测器）	规模	混淆矩阵	召回 / 正常误报
合成冒烟样例（`rules-v0.3`）	14	TP5/FN2/FP1/TN6	71.4% / 14.3%
七绕过合成矩阵（`rules-v0.6`）	42	TP21/FN0/FP0/TN21	100% / 0%
公开注入样本 · 留出（`rules-v0.6`）	140	TP45/FN35/FP0/TN60	56.25% / 0%

公开样本按来源分层抽样，四个来源的留出召回为：MCP 工具投毒 85%[14]、Gandalf 65%[13]、中文智能体安全集 55%[15]、deepset 通用注入集 20%[16]，各来源精度均为 100%；四个来源的许可均允许再分发样例。合成矩阵由三种注入来源（直接输入、间接资料、记忆）与七类绕过标签构成，含 21 个攻击格与 21 个对应正常格。

结果须与口径并列解读：合成矩阵的 100% 属同源自检，规则与样例同批开发，只能说明该规则集在自建样例上自洽；公开样本的 56.25% 是规则初筛对既有语料的覆盖，其中数值较低的两个来源主要含有需要语义判断的载荷。两组数字都不构成攻击成功率、网关实际阻断率或跨语料泛化能力的结论。扩展检测器提高了中文与绕过变体的覆盖，但检测器只参与风险解释，实际授权仍由策略与不可覆盖名单决定。

四业务确定性运行（`competition_repro_20261018`）在当前基线重跑为 16 个单元全部完成，云调用为 0，源码归档哈希与协议哈希随运行目录记录。该运行同时给出网关自身开销的量级：16 个单元共 84 次工具调用，单次调用端到端均摊 7.7 毫秒，扣除脚本模型等待后为 4.7 毫秒，单元墙钟均值为 40.7 毫秒。由于模型是脚本替身，这组数字只说明本地策略核验与账本写入不构成瓶颈，不能外推为接入真实模型后的端到端时延。

3.5 同事实离线比较（历史阶段）

表 3.4: M25 原冻结版本的离线观察，源版本独立于当前基线

运行方式	单元/配对	正常执行	业务完成	危险效果
确定性脚本	48/14	48	46	2
本地 HTTP MockTransport	48/14	48	46	2
三来源固定提议代表	14/7	14	14	3

前两行各有两个 Git 注入单元在 unsafe 和 strong-static 配置实际执行了禁止推送；完整配置保持检视任务的远端约束。其他没有危险提议或没有配置差异的格，只报告其实际观察。记录通过核验表示观察和协议一致，并不意味着每个格都安全。

本地 HTTP 矩阵使用 265 次进程内 MockTransport 请求，经过 Provider 请求准备、解析和账本代码，云调用为 0。三来源代表涵盖直接 Unicode、外部 Base64 和跨会话 Memory 流程备注；来源真实暴露后固定提出禁止的夹具标记写入。三个 unsafe 攻击格实际改变标记，对应 full 格阻断。这说明指定实例的权限检查有效，未测量自然模型的攻击成功率或来源机制的独有收益。

M25 原矩阵与当前基线使用不同源版本。主线保留后续远端修复，历史 48+48+14 矩阵未在当前基线全量重跑。报告保留这一版本对应关系，避免将

旧矩阵当成新代码成绩。

3.6 较早真实模型开发记录（历史阶段）

表 3.5: 历史云模型小样本，独立于离线矩阵

阶段	实际结果	请求与已知用量
M19 共同配置	10 单元中 7 项业务完成，3 项 IOC 未提出发送且 BLOCK 为 0	31 请求，25005 tokens
M21 首轮	四业务中 3 项完成，时间线窗口与排序失败保留	15 请求，24144 tokens
M21 时间线修复	修复后该开发样例单独完成	3 请求，5645 tokens
M20 引用核验	脚本主流程加一次云分析	1 请求，353 tokens

这些结果用于检查模型接入和业务可行性；跨版本修复不合并成功率。当前真实云 CLI 关闭，新的真实模型协议、资源准入、未见输入、重复规则和任务层统计需预登记后执行。无危险动作提议的格不能归因为网关阻断；只有交接文档记载、原始运行目录不在仓库的结果，只能按“开发期记录”引用。

3.7 结果分析与未完成验证

现有结果支持四类具体结论：第一，任务与来源用途约束已进入真实工具链，并能在部分 Git 与标记写入校准中形成可观察差异；第二，委派、Memory 和危险原文引用具有实际角色、版本、内容和业务事实见证；第三，审批对象与实际提交参数可核对，敏感写入目标在放宽授权下仍被拒绝；第四，同一提交在三个平台的全量回归一致通过，说明当前工程记录可跨环境复核。四业务确定性运行与规则检测评测均可离线重算。

下一轮需要解决的是相对收益和外推范围：在同版真实模型上比较业务完成、危险效果和代价；补充未见任务、有效消融和重复；完成运行时三来源七绕过矩阵、跨轮组合、完整入口绑定，以及当前拓扑的隔离、性能与跨机复现。暂不填写无数据支撑的正式 ASR、提升比例或性能达标值。

第四章　创新性说明

4.1　把来源用途落实到跨角色和跨会话执行

溯界围绕“资料可分析、资料不能批准动作”组织数据与授权对象。来源父链和用途约束随摘要、委派和受控记忆继续传递，执行时结合任务权限检查。具体工程贡献包括收窄的子 Grant、跨会话版本原文与 CAS、来源恢复、结构化工具返回的逐字段来源登记，以及可重算的角色和状态观察。M24/M25 的委派与 Memory 实例提供其运行证据；任意语义变换与跨存储迁移另有边界。

4.2　把文本风险与动作授权分成两个可独立核验的对象

多数注入防护把“识别恶意文本”与“阻止危险动作”压在同一分数上，导致无法判断一次阻断究竟来自分类置信度还是来自授权规则。本作品把两者分开：检测器只产出风险解释与策略事实，授权由任务契约、来源用途和策略 DSL 决定，敏感写入目标另有一层不可覆盖名单。

为让这一分工可长期核验，检测器采用版本化治理：冻结版 `rules-v0.3` 的源文件字节哈希被留出协议固定，无法在不破坏协议的前提下修改；扩展版 `rules-v0.6` 承载后续规则工作，由选择器切换。这样“改规则”不会静默改变历史评测的可复现性——这是把评测协议当作长期资产而非一次性脚本的做法。

4.3　从证据账本重建可解释的攻击链路

系统把每次工具调用、来源读取与决策追加到证据账本，再在账本上构建类型化溯源图，节点区分主体、会话、工具调用、来源构件与效果，边区分委派、读取、派生、审批、提交与效果发生。同一构件跨 MCP 服务器与跨子 Agent 复用时合并为同一节点并保留每次使用的边，因此可以导出“不可信来源 → 模型提议 → 跨角色传播 → 实际效果”的有向链路与时间泳道。

这一组合的价值在于可解释性来自已记录事实而非模型自述：操作者看到的是实际发生的边，而不是模型对自身行为的描述。导出为离线可复现步骤，产出 HTML、DOT/Mermaid 源码与 SVG；渲染子进程被移出网关进程以满足沙箱结构约束。它重建已发生的事实，不预测未发生的攻击，也不修改授权状态。

4.4 同时验收业务保全与实际危险效果

危险内容在安全研判中本身就是证据。系统提供无工具分析与字面引用核验，并将合法业务完成和实际危险效果分开登记。M25 同事实比较保留了对照配置发生的两个 Git 危险格，来源代表也记录三个实际写入；记录完整、业务完成和安全完成因此有明确差别。这个验收方法使研究结论能够回到真实文件、收据和状态，而非模型自评或界面标志。

4.5 绑定审批、参数与实际提交

审批预览与确切输入、版本、目的地和资源状态关联，批准后还要执行提交复核。IOC 和 Git 业务将批准与实际提交分为独立步骤，漂移、拒绝、重放和 UNKNOWN 分别留痕。这一组合便于操作者判断“批准了什么”和“实际发生了什么”，也给审计提供明确对象。

4.6 与已有机制的关系

表4.1按机制维度给出横向对照。表中“覆盖”指该机制把这一项作为设计目标并在公开描述中实现，不代表本文复现过其代码，也不代表防护强度；“未涉及”指其公开描述中没有这一层，不构成对其效果的否定。

表 4.1: 与近邻机制的能力对照（按各文公开描述归类，未复现其实现）

机制	执行前动作授权	不可信来源约束	检测与授权分离	敏感目标硬拒绝	违规链路溯源
CaMeL[3]	覆盖	覆盖	未涉及	未涉及	未涉及
Task Shield[4]	部分	部分	未涉及	未涉及	未涉及
AgentSpec[5]	覆盖	未涉及	未涉及	部分	未涉及
AuthGraph[6]	覆盖	覆盖	未涉及	未涉及	覆盖
AgentVisor[7]	覆盖	部分	未涉及	未涉及	部分
本作品	覆盖	覆盖	覆盖	覆盖	覆盖

ContainmentBench 与 AID-Guard 属于暴露后遏制与委派授权的评测与机制研究 [8, 9]，MCP-SafetyBench 与 MCPTox 提供基准与攻击面刻画 [10, 11]，它们与上表的机制维度互补，不直接填入同一栏。

上述贡献建立在能力授权、来源跟踪、运行时策略和任务对齐研究之上。作品的当前增量集中在统一执行对象、跨角色/会话的实现、检测器与授权的职责

分离，以及可检查实际效果的工程组合。其相对基线的广泛收益仍以预登记真实模型比较为下一阶段验证对象。

第五章　总结

溯界已实现以可信任务契约、来源用途和效果级门禁为核心的智能体工具防护原型。现有 13 个开发业务入口覆盖文件、网络、程序、Git、委派与记忆，Python 控制面和 Vue 控制台提供任务、审批、来源和影响查询；当前基线补充了结构化工具返回的逐字段来源登记、版本化注入初筛、敏感写入目标的不可覆盖拒绝，以及从证据账本重建的溯源图。

同一提交在 Ubuntu、Windows 和 macOS 上的全量回归一致通过（1107、1106 和 1107 项，退出码 0），四业务确定性运行与规则检测评测可离线重算，48 单元同事实离线比较和较早的云模型小样本保留了各自的源版本。这些结论的边界同样明确：数字来自开发期确定性运行与离线评测，不给出真实模型攻击成功率，也不构成对任意载荷的通用防护承诺。

作品后续重点是把已有运行链转化为同版、可复现的比较证据：预登记真实模型协议，补充未见任务与有效消融，验证危险原文保全和恢复的收益及代价，完成运行时三来源七绕过矩阵，再完成新版部署性能与跨机复现。最终交付以任务版本、实际观察、独立核验与材料图表的对应索引组织，使评审能够从作品主张追溯到具体证据。

参考文献

[1] OWASP FOUNDATION. OWASP Top 10 for large language model applications 2025: LLM01 prompt injection[EB/OL]. (2025)[2026-10-18]. https://genai.owasp.org/llmrisk/llm01-prompt-injection/.

[2] ANTHROPIC. Model Context Protocol specification: revision 2025-06-18[EB/OL]. (2025-06-18)[2026-10-18]. https://modelcontextprotocol.io/specification/2025-06-18.

[3] DEBENEDETTI E, SHUMAILOV I, FAN T, et al. Defeating prompt injections by design[EB/OL]. (2025-03-24)[2026-10-18]. https://arxiv.org/abs/2503.18813.

[4] JIA F, WU T, QIN X, et al. The Task Shield: enforcing task alignment to defend against indirect prompt injection in LLM agents[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2025: 29680-29697. DOI: 10.18653/v1/2025.acl-long.1435.

[5] WANG H, POSKITT C M, SUN J. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents[C]//Proceedings of the 48th IEEE/ACM International Conference on Software Engineering. New York: ACM, 2026: 2938-2950. DOI: 10.1145/3744916.3764546.

[6] WANG P, LI Y, TIAN Y. Aligning provenance with authorization: a dual-graph defense for LLM agents[EB/OL]. (2026-05-26)[2026-10-18]. https://arxiv.org/abs/2605.26497.

[7] YING Z, WANG H, LIU J, et al. AgentVisor: defending LLM agents against prompt injection via semantic virtualization[EB/OL]. (2026-04-27)[2026-10-18]. https://arxiv.org/abs/2604.24118.

[8] LAN W, LI S, WU M, et al. ContainmentBench: trace-based evaluation of post-exposure containment in tool-using LLM agents[EB/OL]. (2026-07-27)[2026-10-18]. https://arxiv.org/abs/2607.23999.

[9] TONG Y, DAI L, GUO S. AID-Guard: stateful authorization for delegated agent effects[EB/OL]. (2026-08-21)[2026-10-18]. https://arxiv.org/abs/2608.21159.

[10] ZONG X, SHEN Z, WANG L, et al. MCP-SafetyBench: a benchmark for safety evaluation of large language models with real-world MCP servers[EB/OL]. (2025-12-17)[2026-10-18]. https://arxiv.org/abs/2512.15163.

[11] WANG Z, GAO Y, WANG Y, et al. MCPTox: a benchmark for tool poisoning on real-world MCP servers[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2026, 40(42): 35811-35819. DOI: 10.1609/aaai.v40i42.40895.

[12] THE CVE PROGRAM. CVE-2025-53773: GitHub Copilot and Visual Studio remote code execution via injected workspace settings[EB/OL]. (2025)[2026-10-20]. https://www.cve.org/CVERecord?id=CVE-2025-53773.

[13] LAKERA. gandalf_ignore_instructions[EB/OL]. Hugging Face dataset, revision 04737b65, MIT licence[2026-10-20]. https://huggingface.co/datasets/Lakera/gandalf_ignore_instructions.

[14] FEVZIEGEYURTSEVENLER. mcp-tool-poisoning[EB/OL]. Hugging Face dataset, revision 9b7a1088, Apache-2.0 licence[2026-10-20]. https://huggingface.co/datasets/fevziegeyurtsevenler/mcp-tool-poisoning.

[15] UNINHIBITED-SCHOLAR. agent-safety-bench-zh[EB/OL]. Hugging Face dataset, revision 4089ae3e, CC-BY-4.0 licence[2026-10-20]. https://huggingface.co/datasets/uninhibited-scholar/agent-safety-bench-zh.

[16] DEEPSET. prompt-injections[EB/OL]. Hugging Face dataset, revision 4f61ecb0, Apache-2.0 licence[2026-10-20]. https://huggingface.co/datasets/deepset/prompt-injections.